Object Oriented Programming
École Centrale de Lille, G1/G2 – 2025-2026

Sequence 2 – Part 1 – Scheduling

Mathis Guckert, Gaël Guillot, Maxime Ogier, Frédéric Semet

Context and objectives

The French company Masks&Protection, based in the north of France, is currently facing a high demand for the production of single-use face masks. This year, the demand is really high and the production managers are not sure they will be able to produce all the masks on time. Precisely, the contracts signed with the customers stipulate that the company has to deliver the masks before a given deadline. In case a delivery is performed after the deadline, the company has then to pay huge penalties, proportional to the delay. It is therefore essential to respect as closely as possible the deadlines imposed by the customers.

Masks&Protection decided to hire the best students of the Hauts-de-France region to improve its production planning. The task is to design a software able to provide a production planning capable of managing large quantities of demands.

The objective of this lesson is to design a tool to help Masks&Protection in its work. First, we will focus on the object model to be implemented.

This subject will illustrate the following concepts:

- the notion of interface;
- the management of collections of objects in Java;
- the interface **List** and its implementations (**ArrayList** and **LinkedList**).

Good practices to adopt

- Test your code regularly.
- Always respect the Java naming conventions.
- Always indent your code correctly.
- Choose understandable and relevant names for your classes, attributes, methods, and variables.
- Try to respect as much as possible the encapsulation principle.
- Use appropriate data structures.
- Make the methods short with few indentation levels.
- Do not duplicate pieces of code, but favour reusable code.
- Use the *refactoring* tools of your IDE (right-click then *Refactor* on IntelliJ IDEA).
- Use Javadoc to understand the behaviour of the methods you call.
- When necessary, comment your own code according to the Javadoc format (comment starting with `"/**"` above the attribute and method definitions).

1 Modelling of the production area of Masks&Protection

The production manager, Mister Safety, explains that his job is to plan the production of face masks on the machines of the production area. This involves deciding for each machine the order in which the tasks are performed in order to minimise the production time or costs, while respecting the constraints imposed by the production system.

Precisely, Mister Safety explains that there are n identical machines working in parallel in the production area. A task T_i is an elementary entity of work of processing time (duration) p_i. To fix ideas, a task corresponds to a set of identical face masks to be delivered to the same customer. Moreover, when tasks are assigned to the machines, each task T_i is time stamped with a starting time t_i and a completion time c_i ($c_i = t_i + p_i$). Note that when speaking about time, we are not interesting about a date with day and month, but rather the elapsed time since the starting of the production. Thus, we assume the initial time is 0, and we use the minute as temporal unit. So for example, a time $t = 120$ means 120 minutes after the initial time.

In addition, each task is defined by a deadline d_i, and a unit penalty cost r_i to be paid for each minute of delay with respect to the deadline. When assigned to a machine, the total penalty cost f_i of a task T_i will then be: $f_i = r_i \times \max\{c_i - d_i; 0\}$.

Planning the production consists in scheduling the tasks, that is, to assign the tasks to the machines and to provide the sequence of execution of the tasks on each machine (it is thus also necessary to decide the start time of each task). An example is given in Figure 1. In this example, there are 13 tasks. Task T_4 is assigned to machine M_1, it starts at $t = 3$ and ends at $t = 8$. At time $t = 8$, machine M_1 becomes available and task T_8 starts.

A task can be started on any machine as long as the machine is available. Thus, the start time of a new task on that machine is given by the completion time of the previous task on that same machine. There is no point in

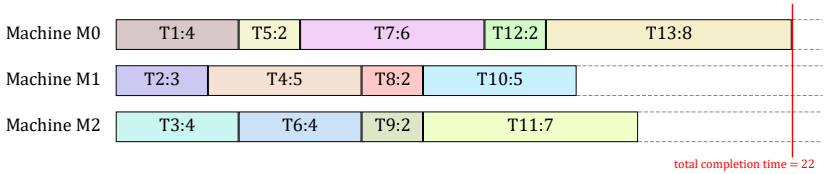

Figure 1: Example of a scheduling on 3 machines. Each task is denoted $T_i : p_i$.

having waiting times. Moreover, Mister Safety explains that a task cannot be interrupted to be continued later.

In this lesson, we will set up the object model required to schedule a set of tasks on the machines of the production area. The UML class diagram for this application is presented in Figure 2.

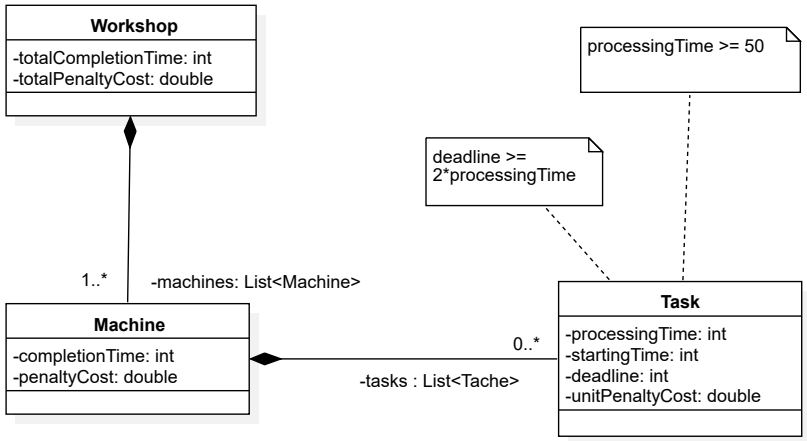

Figure 2: UML class diagram.

1.1 Creation of the Task class

First, we are interested in the representation of a production task. For this purpose, Mister Safety explains that in the production area of Masks&Protection the processing time of a task is at least 50 minutes. Some tasks might not have any deadline: in this case, the deadline is considered as infinite and the unit penalty cost is zero. Moreover, Mister Safety negotiated with the sales department such that the deadline of a task will be at least twice the processing time of the task (so $d_i \geq 2p_i$). Mister Safety explains that the processing time cannot be changed and, he has also obtained from the sales department that deadlines and unit penalty costs cannot be modified. In addition, each task must be characterised by a unique identifier.

Question 1. After opening IntelliJ IDEA, create a new project **Scheduling**, a package **workshop** (which will contain the classes to model the production area), and write a **Task** class which represents a task to be performed by Masks&Protection. Define the attributes of this class **Task**.

Question 2. Add the following three constructors:

- a default constructor;
- a data constructor with the processing time;
- a data constructor with the processing time, the deadline and the unit penalty cost.

Question 3. Complete the **Task** class by adding the necessary getters and setters and by overriding the **toString()** method.

Question 4. Write a main method to test the use of the **Task** class. Be sure to test all possible cases and check that the processing times, deadlines, and unit penalty costs always have consistent values with respect to the specifications.

1.2 Creation of the Machine class

1.2.1 Theory: interfaces

In the previous lesson, we said that an abstract class permit to define common behaviours in a base class, while also imposing the children classes to implement some methods (the ones defined as abstract in the abstract class).

Let us consider an abstract class without any attribute (except if they are declared as **static**), without any constructor, and defining only abstract methods. Then such a class would be equivalent to what is called in object oriented programming an interface. The definition of an interface is similar to the one of a class. The keyword **interface** is used instead of the keyword **class**, like in the following:

```java
public interface I {
   void f(int n); // Signature of a method f
   double g(); // Signature of a method g
}
```

By nature, the methods of an interface are abstracts and public, and so it is not mandatory to mention it with the keywords **abstract** and **public**. Moreover, note that an interface has no constructor and define only abstract methods, hence it is not possible to instantiate an interface, that is, with the previous example it is not possible to write **new I()**.

When defining a class, it is possible to precise in the signature that the class implements a given interface by using the keyword **implements**, like in the following:

```java
public class A implements I {
   // A has to implement methods f and g defined in the interface I
    @Override
    public void f (int a) {
      // Implementation of method f
    }

    @Override
    public double g () {
      // Implementation of method g
    }
}
```

With the Java language, a class can inherit from a single parent class, but a class can implement several interfaces (in such a case, the names of the interfaces are separated by commas). It is a way to manage multiple inheritance with the Java language.

1.2.2 Theory: polymorphism

In object oriented programming, polymorphism is a concept that allow to manipulate an object without necessarily knowing its exact type. For example, in the previous lesson, in the Formula 1 team we were manipulating people in the staff and cars in the fleet without exactly knowing if the cars were Formula 1 or trucks, and if the people were pilots or technicians. To do so, we were using the fact that an object is recognized as an instance of its class, but also as an instance of all the classes in its inheritance hierarchy (parent class, grand parent class, and so on). As an example, an object of type **Pilot** was also of type **Person**, and also of type **Object**.

Moreover, when a class defines a method, all its children classes necessary also have this method, that is the basis of inheritance. But in addition, the children classes have the possibility to override (that is to redefine or modify) the methods inherited by the parent class. This is what we usually do with the **toString()** method defined in the **Object** class. This possibility of overriding inherited methods is the basis of the concept of polymorphism. It is then possible to treat in the same way objects of different types, under the condition that they inherit from the same parent class. This is why for example we can call the **toString()** method on any object without knowing the effective behaviour (the overriding) of that method.

Note that the concept of polymorphism previously described in the context of inheritance also applies in the context of abstract classes and abstract methods. Thus, in the previous lesson, we have been able to call the abstract method **isCompatible(Car c)** in the class **Person** without knowing the exact implementation of that method in the children classes.

Moreover, the concept of polymorphism also applies in the context of interfaces:

- an object is recognized as an instance of each of the interfaces it implements (thus in the previous example an object of type **A** is alo of type **I**);
- if a (non abstract) class implements an interface, we are sure that the methods defined in the interface will be implemented, and then we can call them without knowing the exact type of the object.

1.2.3 Theory: collection of objects

To manipulate a collection of objects with the Java language, the package **java.util** provides a set of classes that allow to manipulate dynamic arrays, sets, linked lists, queues and associative tables. Classes related to dynamic arrays, liked lists, sets and queues all implement an interface named **Collection**.

In fact, collections of objects are manipulated with generic classes that implement the interface **Collection<E>** where **E** represents the type of all elements in the collection. Hence, all elements of a collection are of the same type **E**. Note that with polymorphism, it is possible that elements of the collection are of a subtype of **E**, that is instances of a child class of **E**.

1.2.4 Theory: loop over the elements of a collection

The loop *for...each* allow to easily iterate over a collection **col** of objects of type **E** as follows:

```java
for (E elem : col) {
   // Use of elem
}
```

The variable **elem** successively takes the value of each of the references of the elements in the collection. **HOWEVER**, this loop cannot be used if one wishes to modify the collection using methods such as **remove()** or **add()** while iterating over the elements!

There are also specific objects, called "iterators", which allow to loop over the elements of a collection. An iterator looks like a pointer (like those in C or

C++) without having exactly the same properties. Each collection class has a method named **iterator()** providing a one-way iterator, i.e. an object of a class implementing the **Iterator<E>** interface. It is therefore also possible to loop over a collection **col** of objects of type **E** in the following way:

```
Iterator<E> iter = col.iterator();
while (iter.hasNext()) {
   E elem = iter.next();
   // Use of elem
}
```

The **iterator()** method returns an object pointing the first element of the collection, if it exists. Method **next()** provides the element pointed by **iter** and advances the iterator to the next position. To loop over the same collection several times, the iterator has to be reset by calling the **iterator()** method again.

The interface **Iterator** provides a method **remove()** which removes from the collection the last object returned by the **next()** method.

1.2.5 Theory: list of objects

A list of objects corresponds to an ordered collection of objects. This means that:

- each object has a position in the list, and it is thus possible to access an object by giving its positions: there is a first element, a second one, a third one, and so on;
- a same object can appear several times (at different positions).

It is important to note that when defining a list as an ordered collection does not signify the elements in the list are sorted according to a certain criterion: the notion of ordering is different from the notion of sorting. Moreover, since a list is ordered, there are methods to add or remove elements given as parameter the position where the operation has to be done. Finally, note that since there is no guarantee that elements are sorted, the search of an element by its value (knowing if an object is present in the list) is performed with a time complexity $O(n)$ (n being the size of the list). It is indeed necessary to iterate over all the elements of the list.

In the Java language, lists are classes implementing the **List** interface. The **List** interface extends the **Collection** interface. **List** being an interface, of course it is not possible to instantiate it. In the Java language, there are two main classes that implement the **List** interface: **ArrayList** and **LinkedList**. To know the methods provided by these classes of objects, you can use the Javadoc. Basically, when you want to use a data structure of type list, you have to choose between **ArrayList** and **LinkedList** the one that suits the best.

The **ArrayList** class represents a dynamic array, i.e. an array with variable size. Thus, as for an array, each element can be accessed by its index in a very efficient way (time complexity $O(1)$). Also, note that this class is much more flexible than usual arrays since its size (the number of elements it contains) can vary during the execution (like for any collection of objects). However, to have a fast access to an element at a given index, it is necessary that objects (more precisely their references) are contiguous in the memory (like in a classical array). Hence such a class suffers from a drawback inherent to its nature: adding or removing an object has a time complexity $O(n)$ in the worst case. Thus, **ArrayList** are well suited to perform many direct access to the elements, under the condition that adding and removing an element is not too frequent.

The **LinkedList** class represents a double linked list. The main advantage of this data structure is to allow adding and removing elements while iterating over the list with a temporal complexity of $O(1)$ (this is done with a modification of pointers referring to the previous and next elements). However, the access to an element by its index is not efficient since it requires to iterate over the list, with a temporal complexity of $O(n)$. Note that as a consequence adding or removing an element by its index also has a complexity $O(n)$. Hence, **LinkedList** are well suited to dynamically add or remove elements, under the condition that direct access to elements by their index remain limited.

There is no class implementing the **List** interface that is always the most relevant to use. The choice has to be done by considering the particular characteristics of the expected usage. This observation does not only apply for the classes implementaing the **List** interface, but also for classes implementing the **Set** and **Map** interfaces.

1.2.6 List of task in the Machine class

Masks&Protection has a limited number of machines to perform the tasks. Mister Safety who has been sequencing tasks with his pen and paper for twenty years, explains that when assigning tasks to one of the machines, it is very important to know:

- the current completion time of the machine, i.e. the time instant from which a new task can be started on the machine;
- the penalty costs associated with this machine (that is the sum of the total penalty costs of the tasks currently scheduled on the machine).

Every Friday evening, Mister Safety has to solve a very challenging problem: how to assign tasks to machines in order to minimise the total completion time of the schedule (i.e. the time at which all tasks are completed), and then if possible to minimise the penalty costs for delays.

We will try to help Mister Safety later. For now, Mister Safety explains that to perform his scheduling he initially assigns each machine with an empty *list of tasks* and, then, he progressively adds tasks to each of these lists. On each machine, the tasks are executed in the order in which they appear in the list.

Question 5. In the package **workshop**, add a class **Machine** with its attributes.

A machine being characterised by a list of tasks, you have to choose if you will use an **ArrayList** or a **LinkedList**. You must be able to justify your choice.

Question 6. Add a default constructor to the class **Machine**. Add the necessary getters and setters.

Question 7. Add a method **public boolean addTask(Task t)** which adds task **t** to the list of tasks to be executed on the machine. The method returns **true** if the task has been successfully added, **false** otherwise. The method should also correctly update the state (the value of the attributes) of the machine and task **t**.

The interface **List** has a method **add(Object o)** that adds **o** at the end of the list.
Try to propose a clean code where the modifications related to the task are done using methods of the **Task** class and not directly in the method **addTask(Task t)** of the **Machine** class.

Question 8. Override the **toString()** method. Ensure to make the string easily readable, for example by adding a new line ("\n") before each task in the list. Add a main method to test the **Machine** class.

What is the completion time and penalty to be paid by Masks&Protection if it receives the following tasks (defined by processing time, deadline and unit penalty cost) and schedule them in that order on a single machine? What are the starting time and penalty cost for task **t5**?

```
Task t1 = new Task(150, 300, 2.5);
Task t2 = new Task(140, 400, 1.5);
Task t3 = new Task(50, 200, 2.5);
Task t4 = new Task(85, 200, 1.0);
Task t5 = new Task(75, 160, 0.5);
Task t6 = new Task(80, 500, 1.5);
```

1.3 Creation of the class Workshop

The workshop of Masks&Protection represents the production area, and mainly consists in a limited number of machines on which the tasks are performed. Mister Safety explains that he needs to know the total completion time (the time at which all the tasks are completed) and the total penalty cost (the sum of the penalty costs of the machines).

Question 9. In the **workshop** package, add a **Workshop** class with its attributes.

A workshop being characterised by a list of machines, you have to choose if you will use an **ArrayList** or a **LinkedList**. You must be able to justify your choice.

Question 10. Add a data constructor with the number of machines in the workshop (the workshop should contain at least one machine). Add the necessary getters and setters. Override the **toString()** method. Write a main method to test the **Workshop** class.

Question 11. Add a method **public void updateCriteria()** which computes the scheduling criteria (total completion time and total penalty cost) and updates the corresponding attributes accordingly.

Mister Safety explains that on Friday evening, before leaving for the weekend, he designs the following week's schedule with the following modus operandi.

- The sales manager provides a list of all the tasks to be completed during the next week.
- Mister Safety examines the tasks one by one in the order they are listed.
- Each task is assigned to the machine with the smallest completion time.

Question 12. In the class **Workshop**, add the following methods to implement the modus operandi proposed by Mister Safety:

- a method **private Machine getMachine(int posMachine)** which returns the machine in position **posMachine** of the list of machines (you may call the method **get()** of the interface **List**);
- a method **private boolean addTask(Task t, int posMachine)** which adds task **t** to the machine in position **posMachine** in the list of machines (you may call the method **addTask()** of the class **Machine**); this method returns **true** if the task is successfully added, **false** otherwise ;
- a method **private int getIdMachineSmallestCompletionTime()** which returns an integer indicating the position in the list of machines of the one with the lowest completion time;
- a method **public void scheduleTasks(List<Tasks> tasks)** which schedules the tasks of list **tasks** according to the modus operandi of Masks&Protection, then updates the scheduling criteria (total completion time and total penalty cost).

Question 13. Test the scheduling method for a workshop with two machines on the set of tasks provided in Question 8. What is the total completion time? What is the total penalty cost?

Question 14. If we are interested in minimising the total completion time of the tasks on the machines, what do you think about the modus operandi proposed by Mister Safety? Based on the test performed in the previous question, can you propose a scheduling of the tasks with a better total completion time? What value do you obtain? Propose a task scheduling algorithm that minimises the total completion time.

References

[1] Delannoy, C., *Programmer en Java, Eyrolles, 2014*